

IA Aplicada com LLMs

Aula 05 — Padrões de arquitetura de agentes

Onde paramos

Na aula passada vocês leram uma dúzia de casos reais perguntando, em cada um:

"que padrão é este?"

E responderam com nomes. Hoje os nomes viram código.

E a Aula 03 tinha deixado uma lista

A nota de tool calling terminou assim:

*estado e memória · múltiplas ferramentas · **outros padrões de arquitetura** · MCP · confiabilidade · context engineering dinâmica*

Hoje riscamos quatro delas.

Vocês têm um laço que funciona

```
for passo in range(max_passos):  
    r = client.chat.completions.create(messages=mensagens, tools=DECLARACOES)  
    if not r.choices[0].message.tool_calls:  
        return r.choices[0].message.content  
    ...
```

Por que ele não está pronto?

Três respostas — e são as três notas de hoje

1. **Ele não sabe parar** — `max_passos` é um teto, não um orçamento
2. **Ele não sabe falhar** — uma exceção derruba tudo
3. **Ele não sabe esquecer** — o histórico só cresce

E uma quarta pergunta, anterior a todas

Ele precisava ser um laço?

Este deck é sobre essa pergunta.

E vocês já têm um case escolhido

A pergunta útil em cada padrão de hoje não é "*entendi?*".

"é este que o meu case precisa?"

Roteiro

1. **A linha divisória**, agora operacional
2. **Autópsia** do exercício 03
3. **Cinco padrões** de workflow
4. **A tabela de decisão**
5. **Composição** — os padrões se encaixam

Parte 1

A linha divisória

A definição (Aula 01)

Workflow: o caminho está no código.

Agente: o caminho é decidido em execução pelo modelo.

Boa para prova. Ruim para decidir às três da tarde.

A versão operacional

Você consegue desenhar o fluxograma antes de rodar?

Sim → desenhe, e escreva o fluxograma em Python.

Não → aí existe motivo para autonomia.

Três perguntas de apoio

	Sim	Não
O nº de passos é conhecido?	workflow	agente
A ordem depende de dados de execução?	agente	workflow
O erro é caro ou irreversível?	reduza a autonomia	tolerável

A terceira não é sobre a tarefa — é sobre a **consequência do erro**.

O que custa subir no espectro

Nível	Chamadas	Previsível	Depurável
Prompt único	1	sim	sim
Chain	N fixo	sim	sim
Roteador	1 + rota	sim	sim
Agente	?	não	só com trace
Multi-agente	??	não	muito difícil

A coluna que dói é a última.

Parte 2

Autópsia do exercício 03

O melhor workflow disfarçado de agente que temos

...foi escrito por vocês, na semana passada.

Etapa	Veredito
1. Coleta	workflow — a parada é um <code>if</code>
2. Consulta	workflow — sempre, sempre nessa posição
3. Raciocínio	decisão — mas uma, não uma sequência
4. Ação	workflow condicional
5. Consulta final	workflow — sempre por último

Quatro das cinco etapas estavam no código

A sequência estava **numerada no enunciado**.

Foi um erro? Não — o objetivo era o **mecanismo**.

Mas em produção, a versão workflow ganharia em tudo:
menos chamadas, sem pular a consulta final, testável etapa a etapa.

Quando o enunciado vem numerado, o código também deveria vir.

Parte 3

Os cinco padrões

Building Effective Agents (Anthropic, 2024)

1. Prompt chaining com portão

```
entrada —> [ LLM 1 ] —> portão —> [ LLM 2 ] —> saída
                        |
                        — falhou —> não segue adiante
```

O portão é código determinístico.

Quando a regra cabe em Python, ela vai em Python:
mais barato, mais confiável, não alucina.

1. Prompt chaining — quando não usar

- Etapas que **não** são independentes de verdade (cada elo é uma chance de perder informação)
- Número de etapas que **depende da entrada**
→ isso não é chain, é agente

2. Roteador

```
entrada —> [ classificador ] —> rota A: código puro (sem LLM)
                        |
                        | —> rota B: outro prompt
                        |
                        | —> rota C: humano
                        |
                        | —> nenhuma —> fila de revisão
```

Tecnicamente: **saída estruturada com um enum** (Aula 02, §7).

2. Roteador — os dois pontos que separam bom de ruim

A rota `nenhuma` é obrigatória.

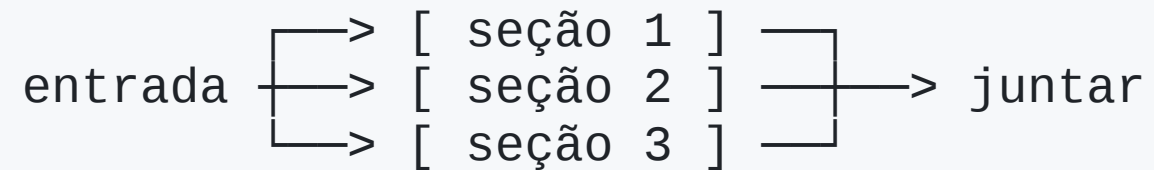
Sem escape, o modelo escolhe uma opção que não serve — com confiança.

A rota mais valiosa é a que não chama o modelo.

Se 70% caem em `aprovar_por_regra`, o sistema ficou 70% mais barato.

3. Paralelização — são duas coisas

Sectioning — subtarefas **independentes**, que você conhece



Voting — a *mesma* tarefa N vezes, maioria decide
(a self-consistency da Aula 03, virada arquitetura)

3. Voting: a divergência é informação

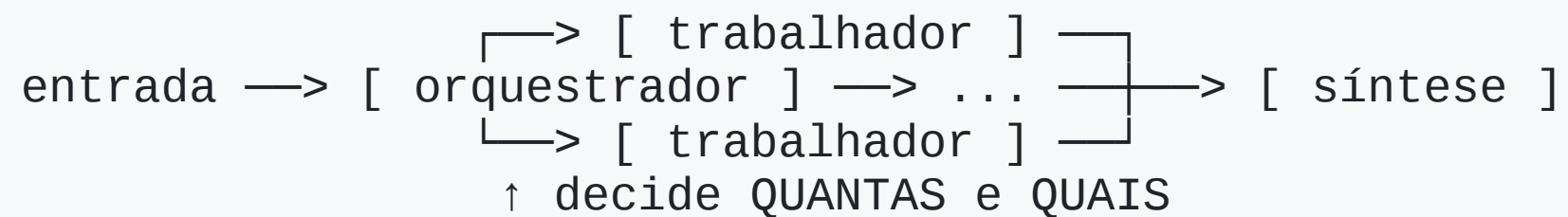
```
if votos < n // 2 + 1:  
    return {"rota": "nenhuma", "motivo": "sem maioria"}
```

Três vereditos diferentes **não** significam "escolha um".

Significam: **este caso é difícil** — provavelmente é caso de humano.

É o sinal de incerteza mais barato que existe. Não descarte.

4. Orquestrador-trabalhador



Diferença para o *sectioning*, e é toda a diferença:
as subtarefas não são conhecidas de antemão.

4. Primeiro padrão com autonomia real

```
if len(plano["subtarefas"]) > MAX_SUBTAREFAS:  
    raise PlanoGrandeDemais(len(plano["subtarefas"]))
```

Um orquestrador sem teto é **uma conta aberta assinada por um modelo.**

Na dúvida entre os dois: prefira *sectioning*.

5. Avaliador-otimizador

```
entrada —> [ gerador ] —> candidato —> [ avaliador ]
      ↑
      | crítica —————|
      |                  | reprovou
      |                  | aprovou —> saída
```

Formalizado como **Reflexion** (Shinn et al., 2023).

5. Duas armadilhas — e aparecem sempre

Sem critério escrito, o avaliador vira elogio.

```
{"nota": 8} # inútil  
{"cita_regra": false, "o_que_corrigir": "..."} # útil
```

Sem teto de rodadas, o par oscila.

E o retorno tem que **dizer** que saiu por teto, não por aprovação.

A tabela de decisão — os previsíveis

Padrão	Custo	Não usar quando
Prompt único	1	precisa de validação por outra chamada
Chaining + portão	N fixo	etapas interdependentes
Roteador	1 + rota	rotas quase iguais
Sectioning	1 por seção	partes dependentes
Voting	3–5	tarefa aberta

Leia de cima para baixo. **Pare no primeiro que resolve.**

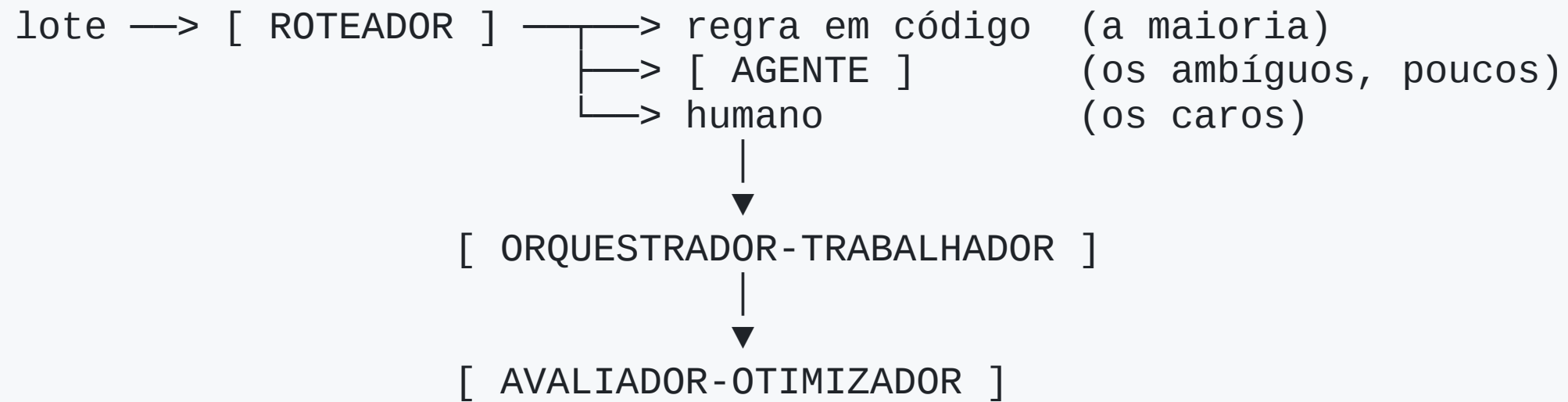
A tabela de decisão — os que exigem limite

Padrão	Custo	Não usar quando
Orquestrador	? — exige teto	you can't list subtasks
Avaliador	2 por rodada	no written criterion
Agente	? — exige orçamento	the flowchart exists

A partir daqui você não sabe quantas chamadas serão.

É por isso que estes três — e só estes — precisam de um número que você escreve **antes** de rodar.

Os padrões se compõem



Este é o desenho do exercício de hoje.

O exemplo que fecha o deck

1.000 despesas, 15% violam a política:

Arquitetura	Chamadas
Agente para tudo	~4.000
Chain fixa	1.150
Roteador + regra em código	450

Nove vezes menos. E os 70% resolvidos por regra são **mais** confiáveis — regra não alucina.

Síntese

- Antes de melhorar o laço: **a tarefa precisava de um laço?**
- A pergunta é uma: **dá para desenhar o fluxograma antes de rodar?**
- Subir de nível custa dinheiro, previsibilidade e **capacidade de depurar**
- No roteador, a rota mais valiosa é a que **não chama o modelo**
- *Sectioning* × orquestrador: **quem define as subtarefas?**
- Avaliador sem critério produz elogio

Autonomia é recurso escasso. Gaste onde ela compra alguma coisa.